

Module 3 Study Guide: Graph Theory Refresher

Before an AI agent can navigate a complex 3D game world, it must understand mathematically what that world is. It does not see textures, lighting, or 3D models. It sees a **Graph**, a mathematical set of nodes and edges. Abstracting level design into a graph permits the proper implementation of pathfinding. If any of this material feels unfamiliar, immediately consult the open-access literature: [Intro to Graph Theory](#).

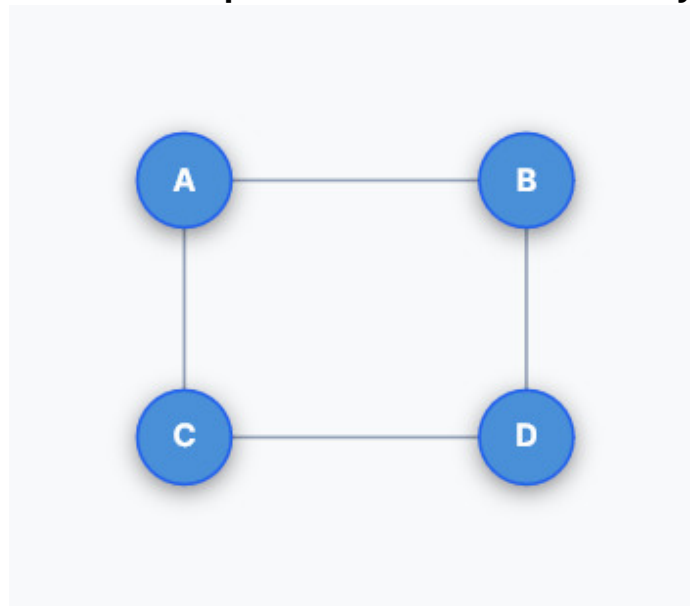
Block 1: Core Graph Theory & Representations

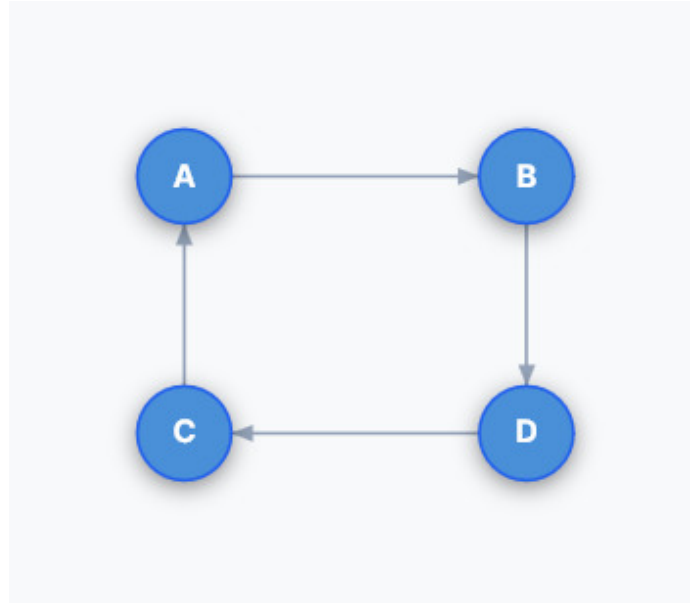
A graph $G = (V, E)$ represents an environment using a set of **Nodes** (Vertices, V) and **Edges** (E). The properties of these edges dictate how our search algorithms must behave.

1. Undirected vs. Directed Graphs

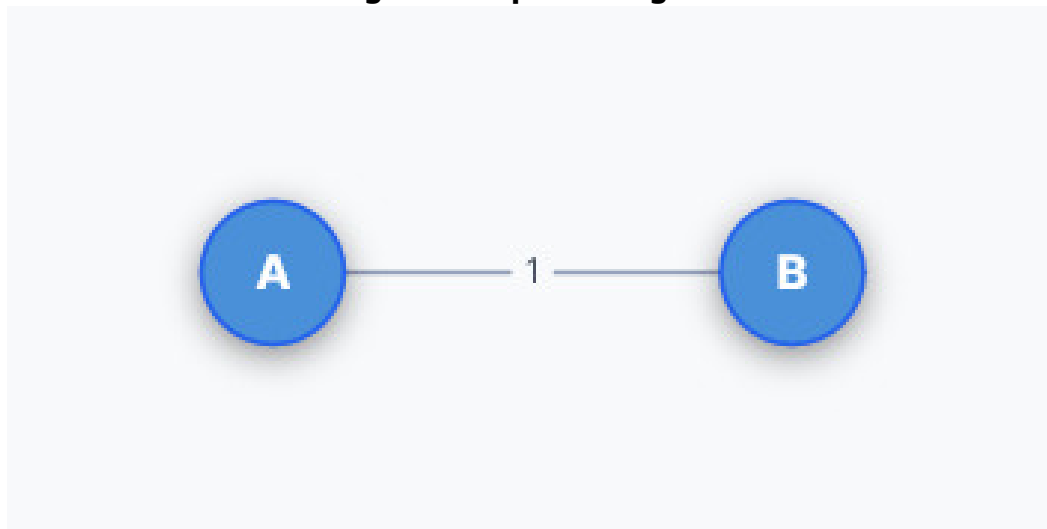
In an **Undirected Graph**, an edge between A and B implies bidirectional movement. In a **Directed Graph** (digraph), movement is restricted to the direction of the edge (e.g., a one-way drop in a platformer).

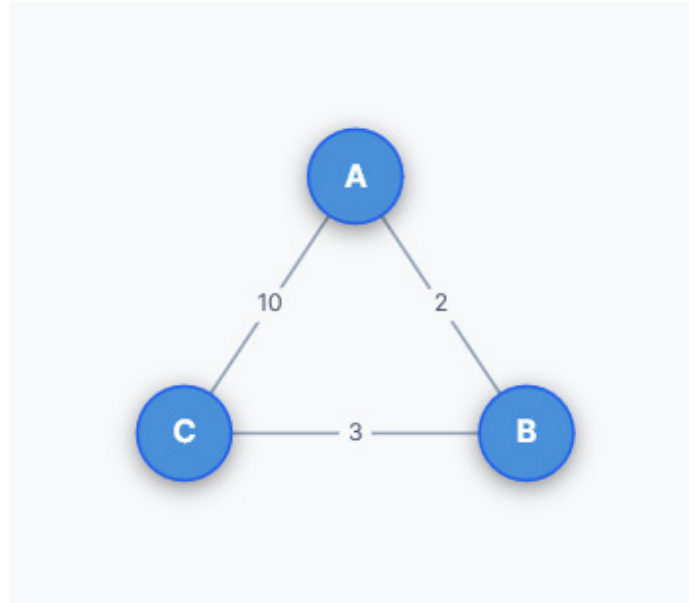
Undirected Graph: Movement is valid both ways



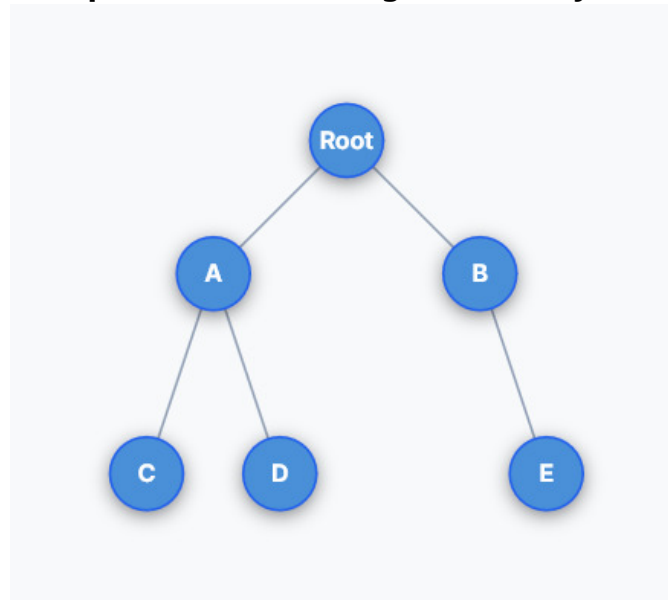
Directed Graph: Movement follows the arrows**2. Unweighted vs. Weighted Graphs**

An **Unweighted Graph** assumes the cost of traversing any edge is equal (usually 1). A **Weighted Graph** assigns a specific cost to each edge, representing distance, terrain difficulty (e.g., mud vs. asphalt), or danger.

Unweighted Graph: All edges cost 1

Weighted Graph: Path A->C is faster via B than going directly**3. Trees**

A **Tree** is a specific type of undirected graph that is connected and acyclic (contains no loops). In search spaces, algorithms dynamically generate trees to represent the history of possible paths branching from a starting state.

Tree Graph: Hierarchical, single root, no cyclic loops

Block 2: The Graph Data Structure in Memory

Before we search, we must define the environment in memory. How you store a graph drastically impacts time and space complexity.

```
# Python 3.12 type aliases for cleaner hinting
type Node = str
type Graph = dict[Node, list[Node]]

# Standard navigation graph representation (Adjacency List)
nav_graph: Graph = {
    'A': ['B', 'C'],
    'B': ['D', 'E'],
    'C': ['F'],
    'D': [],
    'E': ['F'],
    'F': ['G'],
    'G': []
}
```

Architectural Breakdown:

- **Adjacency Lists vs. Matrices:** We represent the graph as a dictionary where the key is the node, and the value is a list of connected neighbours. This is an **Adjacency List**. In sparse game worlds (like a grid where a tile only has 4 neighbours), this uses significantly less memory ($O(V + E)$) than a 2D array (**Adjacency Matrix**) filled with empty space ($O(V^2)$).
- $O(1)$ **Lookups:** Finding a node's neighbours via `nav_graph.get(node)` is an instantaneous $O(1)$ operation.

Block 3: Uninformed Search algorithms

We examine **Uninformed Search**. These are “blind” algorithms. They know where they are, and they know what the goal looks like, but they have no sense of direction. They must blindly sweep through the graph.

Breadth-First Search (BFS) & The Deque

BFS explores a graph radially, layer by layer, checking all immediate neighbours before moving deeper.

```
from collections import deque

def breadth_first_search(graph: Graph, start: Node, goal: Node) -> list[Node]:
    queue = deque([start])
    visited = {start} # O(1) lookups prevents infinite loops

    while queue:
        path = queue.popleft()
        node = path[-1]

        if node == goal:
            return path

        for adjacent in graph.get(node, []):
            if adjacent not in visited:
                visited.add(adjacent)
                new_path = list(path)
                new_path.append(adjacent)
                queue.append(new_path)

    return []
```

Architectural Breakdown:

- **The collections.deque Mandate:** Look at `queue.popleft()`. If you use a standard Python list and call `pop(0)`, the computer must physically shift every remaining item in memory down one index. That is an $O(N)$ operation. Inside a while loop processing 10,000 nodes, your algorithm degrades to $O(N^2)$. A deque (Double-Ended Queue) allows $O(1)$ popping from the front.
- **The Memory Trade-off:** BFS guarantees it will find the shortest path in an *unweighted* graph. But its memory footprint is massive. Because it expands radially, the queue must store the entire expanding frontier simultaneously.

Depth-First Search (DFS) & The Stack

DFS takes the opposite approach. It dives down a single branch until it hits a dead end, then backtracks.

```
def depth_first_search(graph: Graph, start: Node, goal: Node, path: list[Node] =
    None, visited: set[Node] = None) -> list[Node]:
    if path is None: path = [start]
    if visited is None: visited = {start}

    if start == goal:
        return path

    for adjacent in graph.get(start, []):
        if adjacent not in visited:
            visited.add(adjacent)
            # Recursive dive
            result = depth_first_search(graph, adjacent, goal, path + [adjacent
], visited)
            if result:
                return result
    return []
```

Architectural Breakdown:

- **The Stack (LIFO):** DFS operates on a Last-In, First-Out principle. Here, it is implemented using the Call Stack via recursion.
- **The Danger of Sub-optimality:** DFS is incredibly memory-efficient because it only stores the current branch. However, it is “blind” and stubborn. It might find a path that is 500 steps long just because it happened to check the left branch first, completely ignoring a 5-step path on the right branch. It does *not* guarantee the optimal route.

Block 4: Informed Search Algorithms

Uninformed searches are insufficient for large-scale navigation. **Informed Search** algorithms utilise a heuristic (an educated guess) to guide the search towards the goal, drastically reducing the number of nodes evaluated.

- **Greedy Best-First Search:** Selects the path that appears closest to the goal mathematically, ignoring the cost travelled so far. Fast, but not guaranteed to find the optimal path if obstacles exist.
- **A* (A-Star) Search:** Combines actual cost from the start (g) with the estimated cost to the goal (h). It evaluates $f(n) = g(n) + h(n)$ using a Priority Queue. It is optimal and complete, and is the industry standard for 3D pathfinding.

Block 5: Asynchronous Engine Integration via Generators

When integrating these algorithms into a real-time engine loop (e.g., Pygame), you cannot allow a pathfinding while loop to hijack the main thread, or the game will freeze. We alter the architecture using Python **Generators**.

```
def bfs_stepper(start: Point, goal: Point, grid_w: int, grid_h: int, walls: set[Point]):
    queue = deque([[start]])
    visited = {start}

    while queue:
        path = queue.popleft()
        current = path[-1]

        if current == goal:
            yield path, visited, True # True means finished
            return

        # ... (neighbour expansion logic) ...

        yield path, visited, False # Yield current state for engine rendering
```

Architectural Breakdown:

- **The yield Keyword:** By replacing return with yield, the function becomes a Generator.
- **Handing Back Control:** Every time the algorithm checks a node, it yields its current state back to the engine, pauses its own execution, allows the engine to draw the frame, and then resumes exactly where it left off on the next tick. This is how you build asynchronous debuggers without complex multi-threading.

Glossary

- **Graph:** A mathematical structure used to model pairwise relations between objects. Made up of **Nodes** and **Edges**.
- **Uninformed Search:** A “blind” search algorithm (like BFS or DFS) that has no additional information about states beyond the problem definition.
- **Informed Search:** An algorithm (like A*) that uses heuristics to guide the search towards the goal, optimising performance.
- **Breadth-First Search (BFS):** Explores layer by layer. Uses a **Queue (FIFO)**. Guarantees the shortest path in unweighted graphs but has a massive memory footprint.
- **Depth-First Search (DFS):** Plunges deep along a single branch before backtracking. Uses a **Stack (LIFO)**. Memory efficient, but does *not* guarantee the shortest path.
- **Deque (Double-Ended Queue):** Allows elements to be added or removed from either the front or the back in $O(1)$ time. Mandatory for BFS in Python.
- **$O(1)$ vs $O(N)$ Time Complexity:** $O(1)$ means an operation takes constant time. $O(N)$ means the time scales linearly with the data size.
- **Generator / Yield:** Allows a function to return an intermediate value and pause its execution state. Crucial for slicing heavy algorithms across multiple frames to prevent game loop freezing.